

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO BÀI TẬP LỚN
CÁC HỆ THỐNG PHÂN TÁN
ĐỀ TÀI: PHÁT TRIỂN HỆ THỐNG CHAT NGANG HÀNG P2P
(PEER-TO-PEER CHAT SYSTEM)

Giảng viên hướng dẫn: Kim Ngọc Bách

Nhóm lớp: 01

Nhóm bài tập lớn: 09

Mã sinh viên	Họ và tên
B22DCCN353	Nguyễn Văn Huân
B22DCCN077	Bùi Huy Bích
B22DCCN556	Lê Thanh Nam

Hà Nội, 2026

MỤC LỤC

1. Giới thiệu	4
2. Mục tiêu và phạm vi	5
3. Kiến trúc hệ thống	6
3.1 Kiến trúc hybrid P2P	6
3.2 Peer node vừa là client vừa là server	7
3.3 Bootstrap Server	7
3.4 Admin UI	8
3.5 Peer Web UI	8
4. Cơ chế giao tiếp giữa các thành phần	8
5. Giao thức trao đổi thông điệp	9
6. Peer discovery	10
7. Chat trực tiếp, chat nhóm và broadcast	11
8. Truyền file giữa các peer	13
9. Xử lý peer offline và truyền tin đáng tin cậy	15
10. Xử lý lỗi và tính chịu lỗi	16
11. Kiểm thử và demo hệ thống	17
11.1 Kiểm thử	17
11.2 Demo hệ thống	21
<i>a, Đăng ký peer mới</i>	<i>21</i>
<i>b, Giao diện chính của hệ thống</i>	<i>22</i>
1. <i>Giao tiếp giữa các peer (gửi tin nhắn trực tiếp):</i>	<i>23</i>
2. <i>Giao tiếp thông qua group</i>	<i>24</i>
3. <i>Giao tiếp gửi tin nhắn qua broadcast</i>	<i>25</i>
4. <i>Chức năng gửi file (mở rộng)</i>	<i>26</i>
12. Đánh giá	28
12.1 Ưu điểm	28
12.2 Hạn chế	28
12.3 Hướng phát triển	29
13. Kết luận	30

DANH MỤC HÌNH ẢNH

<i>Hình 3.1: Kiến trúc tổng thể của hệ thống.....</i>	<i>6</i>
<i>Hình 6.1: Quy trình peer discovery thông qua bootstrap sever.....</i>	<i>10</i>
<i>Hình 7.1: Luồng gửi tin nhắn trực tiếp giữa hai peer.....</i>	<i>12</i>
<i>Hình 7.2: Cơ chế gửi tin nhắn nhóm bằng cách gửi trực tiếp đến từng thành viên.</i>	<i>12</i>
<i>Hình 7.3: Cơ chế broadcast lan truyền trong mạng peer đã biết.</i>	<i>13</i>
<i>Hình 8.1: Quy trình truyền file trực tiếp giữa hai peer.....</i>	<i>14</i>
<i>Hình 9.1: Cơ chế lưu và nhận lại tin nhắn khi peer đích offline.....</i>	<i>15</i>
<i>Hình 11.1 Giao diện đăng ký peer mới.</i>	<i>21</i>
<i>Hình 11.2 Hệ thống thông báo lỗi khi peer mới trùng tên hoặc port của peer cũ.</i>	<i>21</i>
<i>Hình 11.3: Giao diện trang chủ của hệ thống chat P2P.....</i>	<i>22</i>
<i>Hình 11.4: Giao diện gửi tin nhắn trực tiếp.....</i>	<i>23</i>
<i>Hình 11.5: Message Log của alice gửi cho bob (bên trái) và gửi cho lucas(bên phải).</i>	<i>23</i>
<i>Hình 11.6: Message Log bob (bên trái) và lucas(bên phải)</i>	<i>23</i>
<i>Hình 11.7: Giao diện gửi tin nhắn qua group.....</i>	<i>24</i>
<i>Hình 11.8: Giao diện group đối với các peer mới.</i>	<i>24</i>
<i>Hình 11.9: Peer alice gửi tin nhắn đến group Big city.</i>	<i>25</i>
<i>Hình 11.10: Message log của các peer trong group Big city.</i>	<i>25</i>
<i>Hình 11.11: Hệ thống thông báo đã có người rời nhóm trong message log.</i>	<i>25</i>
<i>Hình 11.12: Giao diện gửi tin nhắn qua broadcast.</i>	<i>25</i>
<i>Hình 11.13: peer alice gửi tin nhắn thông qua broadcast.</i>	<i>26</i>
<i>Hình 11.14: các peer trong hệ thống đồng loạt nhận được tin nhắn.....</i>	<i>26</i>
<i>Hình 11.15: Giao diện gửi file qua tin nhắn trực tiếp.</i>	<i>26</i>
<i>Hình 11.16: Message log thông báo peer alice đã gửi file cho lucas.....</i>	<i>27</i>
<i>Hình 11.17: Giao diện của peer lucas khi nhận file.</i>	<i>27</i>

1. Giới thiệu

Trong các hệ thống phân tán, vấn đề giao tiếp giữa nhiều nút mạng luôn là một nội dung trung tâm. Mỗi nút có thể có trạng thái riêng, có thể xuất hiện hoặc rời khỏi hệ thống trong quá trình chạy, đồng thời việc truyền dữ liệu phải được thiết kế sao cho không phụ thuộc quá mức vào một điểm trung tâm duy nhất. Với bài toán chat nhiều người dùng, mô hình client-server truyền thống thường đặt server vào vai trò trung gian nhận, xử lý và chuyển tiếp toàn bộ thông điệp. Cách tiếp cận này thuận tiện cho quản trị, nhưng cũng làm server trở thành điểm tập trung tải và điểm lỗi quan trọng.

Mô hình ngang hàng, hay Peer-to-Peer, tiếp cận vấn đề theo hướng phân tán hơn. Trong mô hình này, mỗi peer không chỉ là client gửi yêu cầu mà còn là server có khả năng lắng nghe kết nối đến từ các peer khác. Khi các peer đã biết địa chỉ mạng của nhau, thông điệp có thể được truyền trực tiếp giữa hai đầu mà không cần đi qua một server relay. Đặc điểm này giúp mô hình P2P phù hợp để minh họa các khái niệm của hệ thống phân tán như khám phá nút, trạng thái online/offline, truyền thông trực tiếp, xác nhận nhận tin, xử lý lỗi kết nối và khả năng tiếp tục hoạt động một phần khi thành phần điều phối tạm thời không khả dụng.

Tuy nhiên, một hệ thống P2P thực tế vẫn cần cơ chế khởi tạo để các peer tìm thấy nhau. Nếu một peer mới tham gia mạng nhưng chưa biết địa chỉ của bất kỳ peer nào khác, nó không thể tự thiết lập kết nối trực tiếp. Vì vậy đề án sử dụng một Bootstrap Server làm thành phần hỗ trợ khám phá peer. Bootstrap Server trong hệ thống này không đóng vai trò chuyển tiếp tin nhắn online và không truyền nội dung file. Thành phần này chủ yếu ghi nhận peer đang tham gia, cung cấp danh sách peer, quản lý metadata nhóm, theo dõi trạng thái thông qua heartbeat và lưu hàng đợi tin nhắn văn bản khi peer đích offline.

Do đó, hệ thống được thiết kế theo mô hình hybrid P2P. Bootstrap Server tồn tại để hỗ trợ điều phối và khám phá, còn đường truyền dữ liệu chính giữa các người dùng vẫn là TCP trực tiếp giữa các peer. Cách thiết kế này vừa giữ được tính rõ ràng của mô hình P2P trong phạm vi đề án, vừa tránh làm bài toán trở nên quá phức tạp như các mạng P2P hoàn toàn phi tập trung.

2. Mục tiêu và phạm vi

Mục tiêu chính của đồ án là xây dựng một hệ thống chat ngang hàng có thể chạy nhiều peer độc lập, trong đó mỗi peer có thể đăng ký với Bootstrap Server, khám phá các peer khác, gửi tin nhắn trực tiếp, tham gia nhóm, broadcast tin nhắn và truyền file qua kết nối TCP trực tiếp. Hệ thống cũng cần có giao diện web cho peer để thao tác thuận tiện và giao diện admin để quan sát trạng thái runtime của Bootstrap Server.

Các chức năng đã được triển khai trong mã nguồn hiện tại gồm: đăng ký peer bằng username, host và port; peer lắng nghe TCP trên port riêng; gửi và nhận tin nhắn trực tiếp có ACK; tạo, tham gia, rời nhóm và gửi tin nhắn nhóm; broadcast tới các peer đã biết; truyền file trực tiếp, truyền file theo nhóm và truyền file broadcast; lưu file nhận vào thư mục cục bộ; lưu lịch sử chat dạng JSONL; hàng đợi tin nhắn văn bản offline thông qua Bootstrap Server; hàng đợi offline cục bộ khi Bootstrap không khả dụng; heartbeat để cập nhật trạng thái peer; Peer Web UI bằng Flask; Admin UI bằng Flask; và các kiểm thử tự động cho nhiều tình huống chính.

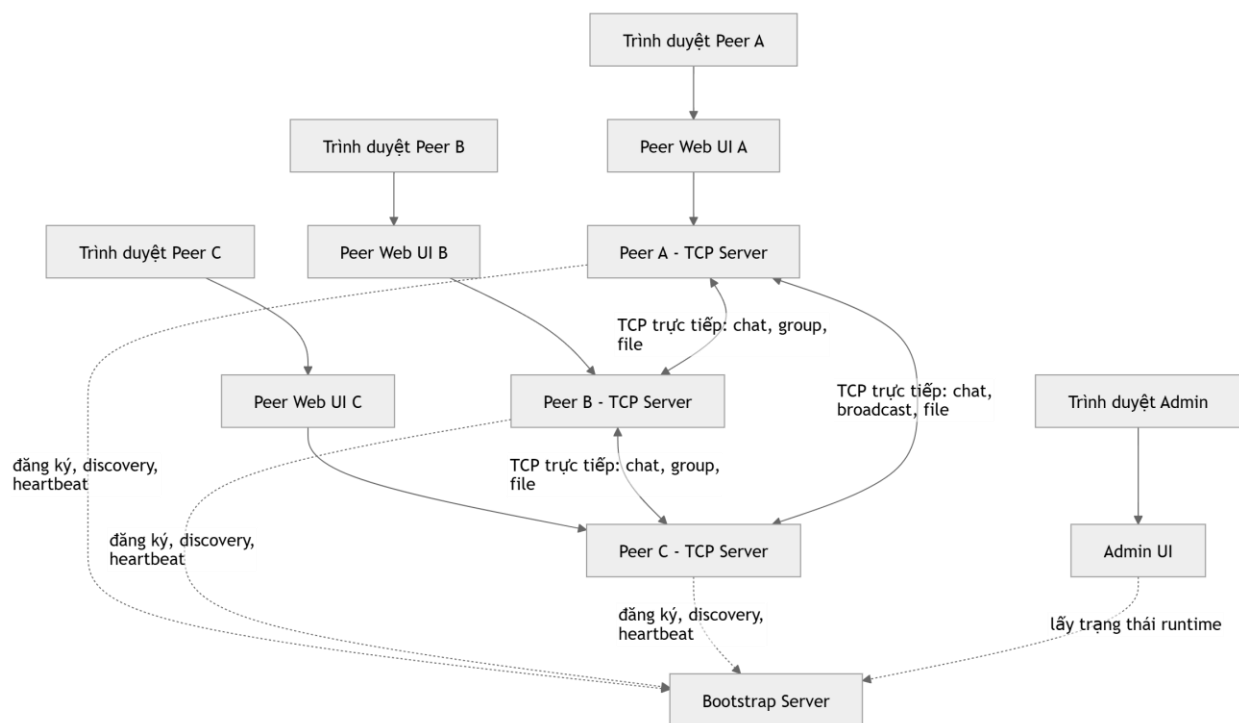
Phạm vi của đồ án được giới hạn trong môi trường local hoặc mạng LAN. Hệ thống chưa triển khai NAT traversal, chưa giải quyết bài toán peer nằm sau firewall hoặc mạng Internet phức tạp, chưa có cơ chế xác thực người dùng mạnh, chưa có phân quyền bảo mật đầy đủ và chưa sử dụng cơ sở dữ liệu để lưu bền vững trạng thái Bootstrap. Dữ liệu registry, group và offline queue của Bootstrap chủ yếu tồn tại trong bộ nhớ trong một phiên chạy. Mã nguồn có cơ chế ghi file storage/peers.json khi khởi tạo snapshot xong, nhưng hàm persist sau các thay đổi hiện không ghi lại đầy đủ trạng thái. Vì vậy không nên mô tả hệ thống như một hệ thống có database hoặc persistence hoàn chỉnh.

Về bảo mật, hệ thống có sử dụng FernetCipher cho nội dung tin nhắn văn bản nếu có khóa hợp lệ từ biến môi trường hoặc file storage/fernet.key. Tuy nhiên đây là mã hóa đối xứng dùng chung khóa trong phạm vi ứng dụng, không phải mô hình mã hóa đầu cuối với khóa riêng cho từng peer hoặc từng cuộc hội thoại. Nội dung file truyền qua TCP hiện không được mã hóa ở tầng ứng dụng. Do đó báo cáo chỉ xem đây là cơ chế mã hóa cơ bản cho payload text, không đánh giá như một giải pháp bảo mật mạnh.

3. Kiến trúc hệ thống

Hệ thống gồm ba nhóm thành phần chính: Bootstrap Server, các Peer Node và các giao diện web. Bootstrap Server giữ vai trò tracker và điều phối metadata. Peer Node là thực thể giao tiếp ngang hàng, vừa gửi dữ liệu đến peer khác vừa lắng nghe dữ liệu từ peer khác. Peer Web UI hỗ trợ người dùng thao tác với một peer cụ thể thông qua trình duyệt, còn Admin UI hỗ trợ quan sát trạng thái của Bootstrap Server.

Kiến trúc tổng thể có thể mô tả như sau:



Hình 3.1: Kiến trúc tổng thể của hệ thống

3.1 Kiến trúc hybrid P2P

Điểm quan trọng của kiến trúc là tách riêng phần quản lý hệ thống và phần truyền dữ liệu giữa các peer. Control plane bao gồm các thao tác đăng ký, khám phá peer, cập nhật heartbeat, quản lý nhóm và xếp hàng tin nhắn offline. Các thao tác này đi qua Bootstrap Server vì chúng cần một điểm tham chiếu chung để nhiều peer cùng thống nhất về trạng thái hiện tại của mạng.

Data plane là luồng dữ liệu chính của người dùng: tin nhắn online và nội dung file. Trong hệ thống hiện tại, dữ liệu này được truyền trực tiếp giữa peer gửi và peer nhận bằng TCP socket. Khi Alice gửi tin cho Bob, Alice không gửi nội dung tin nhắn

đến Bootstrap để Bootstrap chuyển tiếp. Thay vào đó, Alice dùng thông tin host và port của Bob đã khám phá được để mở kết nối TCP trực tiếp đến Peer Server của Bob.

Cách phân tách này làm rõ ý nghĩa của mô hình hybrid P2P. Bootstrap vẫn là thành phần quan trọng khi peer mới tham gia hoặc khi cần đồng bộ metadata, nhưng nó không nằm trên đường truyền nội dung chat online. Sau khi các peer đã khám phá và lưu cache thông tin của nhau, việc truyền tin trực tiếp vẫn có thể tiếp tục trong phạm vi các peer đã biết, ngay cả khi Bootstrap Server tạm thời không hoạt động.

3.2 Peer node vừa là client vừa là server

Mỗi peer trong hệ thống là một tiến trình có hai năng lực đồng thời. Khi đóng vai trò server, peer mở một TCP socket riêng thông qua PeerServer để nhận kết nối từ peer khác. Khi đóng vai trò client, peer dùng PeerClient để kết nối đến host và port của peer đích, gửi thông điệp và chờ phản hồi ACK.

Lớp PeerNode đóng vai trò điều phối các năng lực này. Nó quản lý PeerServer, PeerClient, PeerDiscovery, MessageHandler, vòng lặp heartbeat, cache peer, cache group, logic gửi chat, logic broadcast, logic truyền file và hàng đợi offline. Cách tổ chức này giúp mỗi peer có một điểm điều khiển thống nhất, nhưng vẫn tách các trách nhiệm kỹ thuật thành những module nhỏ hơn.

3.3 Bootstrap Server

Bootstrap Server được triển khai như một TCP server có khả năng xử lý cả request protocol dạng JSON qua socket và một số HTTP API đơn giản. Với các peer, Bootstrap chủ yếu được truy cập thông qua PeerDiscovery. Với Admin UI, Bootstrap được truy cập qua các HTTP endpoint phục vụ dữ liệu quan sát.

Thành phần lõi phía sau Bootstrap là PeerRegistry. Registry lưu trong bộ nhớ danh sách peer, trạng thái online/offline/disconnected, port đã được reserve, metadata group và danh sách tin nhắn offline dạng text. Khi peer heartbeat đều đặn, Bootstrap cập nhật thời điểm last_seen. Nếu một peer đang online nhưng không heartbeat quá ngưỡng cấu hình, registry đánh dấu peer đó là disconnected.

Bootstrap không được thiết kế như một message broker. Khi peer online, Bootstrap không nhận nội dung chat để chuyển tiếp. Với file, Bootstrap cũng không lưu hoặc relay nội dung file. Chỉ trong trường hợp tin nhắn văn bản không gửi được và Bootstrap còn online, peer gửi có thể yêu cầu Bootstrap lưu một bản ghi offline queue để peer đích lấy lại khi reconnect hoặc khi gọi fetch pending messages.

3.4 Admin UI

Admin UI là ứng dụng Flask độc lập dùng để quan sát Bootstrap Server. Giao diện này hiển thị các thông tin runtime như số peer, số peer online/offline/disconnected, số group, hàng đợi offline và danh sách peer. Admin UI không phải nơi phát sinh dữ liệu nguồn. Nó proxy request tới các API admin của Bootstrap Server và hiển thị kết quả.

Khi Bootstrap không reachable, Admin UI trả về trạng thái offline và hiển thị các bảng dữ liệu là không khả dụng. Điều này phù hợp với vai trò giám sát: Admin UI chỉ quan sát trạng thái Bootstrap, không thay thế Bootstrap và không tham gia truyền tin giữa peer.

3.5 Peer Web UI

Peer Web UI là ứng dụng Flask đại diện cho một phiên làm việc của peer trên trình duyệt. Trước khi đăng ký, giao diện hiển thị form nhập username, peer host, peer TCP port, bootstrap host và bootstrap port. Sau khi đăng ký thành công, Web UI tạo một PeerNode, khởi động TCP server của peer, đăng ký với Bootstrap, tải tin nhắn offline đang chờ, bắt đầu heartbeat và bắt đầu vòng retry offline.

Giao diện peer hỗ trợ các thao tác chính: xem danh sách peer đã biết, gửi tin nhắn trực tiếp, tạo nhóm, join hoặc leave group, gửi tin nhắn nhóm, broadcast, gửi file trực tiếp, gửi file theo nhóm, gửi file broadcast, xem log tin nhắn runtime và xem danh sách file đã nhận. Các log hiển thị trong Web UI là log phục vụ người dùng và được giữ trong bộ nhớ của phiên UI, không phải toàn bộ dữ liệu debug của hệ thống.

4. Cơ chế giao tiếp giữa các thành phần

Quy trình giao tiếp bắt đầu khi một peer đăng ký với Bootstrap Server. Peer gửi username, host và TCP port mà nó sẽ lắng nghe. Bootstrap kiểm tra tính hợp lệ của thông tin, ghi nhận peer vào registry và trả về danh sách các peer khác. Từ thời điểm đó, peer có thể lưu danh sách này vào cache cục bộ `known_peers`.

Sau đăng ký, peer định kỳ gửi heartbeat đến Bootstrap. Heartbeat giúp Bootstrap biết peer còn hoạt động và cập nhật trạng thái online. Khi peer dừng đúng cách, nó gửi thông điệp disconnect để Bootstrap đánh dấu offline. Nếu peer ngừng đột ngột, Bootstrap không nhận disconnect, nhưng cơ chế stale detection sẽ chuyển trạng thái từ online sang disconnected sau một khoảng thời gian không có heartbeat.

Khi người dùng gửi tin nhắn trực tiếp, peer gửi tìm thông tin peer đích trong cache. Nếu chưa có, nó cố gắng refresh danh sách từ Bootstrap. Nếu tìm được host và port của peer đích, peer gửi mở kết nối TCP trực tiếp đến peer nhận. Peer nhận xử lý thông điệp, lưu lại nếu hợp lệ và trả ACK. Bootstrap không nằm trên đường đi của tin nhắn online này.

Đối với nhóm, Bootstrap chỉ lưu metadata nhóm: tên nhóm, owner, danh sách thành viên và thời điểm tạo. Khi gửi tin nhắn nhóm, peer gửi lấy danh sách thành viên từ cache hoặc Bootstrap, sau đó gửi thông điệp group trực tiếp đến từng thành viên còn lại. Vì vậy người gửi sẽ gửi trực tiếp tin nhắn đến từng thành viên trong nhóm, không phải một kênh group tập trung tại server.

Broadcast được thực hiện bằng cách peer gửi lần lượt thông điệp đến các peer đã biết trong mạng. Peer gửi tạo thông điệp broadcast, đánh dấu message id đã thấy, rồi gửi tới các peer online trong cache. Khi một peer nhận broadcast mới, nó lưu tin, tránh xử lý trùng thông qua tập `seen_message_ids`, sau đó có thể forward tiếp đến các peer khác đã biết, trừ peer gửi trước đó, peer nguồn và chính nó. Cơ chế này minh họa truyền lan trong mạng P2P, nhưng chưa có TTL hoặc thuật toán định tuyến tối ưu.

5. Giao thức trao đổi thông điệp

Giao thức của hệ thống được xây dựng trên JSON object mã hóa UTF-8, mỗi thông điệp kết thúc bằng ký tự newline để phân tách frame trên TCP stream. Mỗi thông điệp hợp lệ đều có loại thông điệp, mã định danh message, thời điểm tạo và các trường dữ liệu tùy theo ngữ cảnh. Cách biểu diễn này giúp hệ thống có thể phân biệt thông điệp đăng ký, tin nhắn chat, phản hồi ACK, heartbeat, thông báo file và lỗi protocol.

Các thông điệp trao đổi giữa peer và Bootstrap bao gồm đăng ký peer, yêu cầu danh sách peer, heartbeat, disconnect, thao tác group và thao tác offline queue. Các thông điệp trao đổi trực tiếp giữa peer với peer bao gồm chat trực tiếp, chat nhóm, broadcast, thông báo hệ thống, metadata truyền file, phản hồi file, kết thúc file, ACK và ERROR.

Một ví dụ ngắn về cấu trúc tin nhắn trực tiếp có thể được biểu diễn như sau:

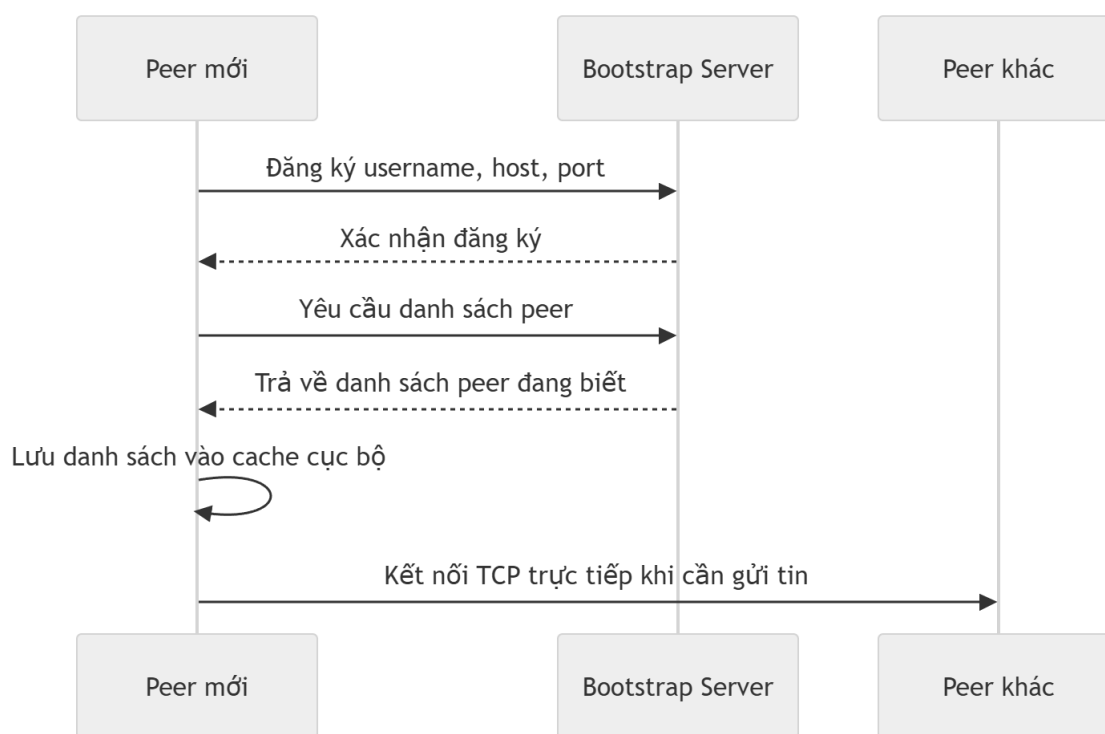
[json]

```
{"type":"CHAT","from":"alice","to":"bob","message":"xin  
chao","message_id":"...","timestamp":"..."}
```

Trong quá trình chạy thực tế, nội dung message có thể là plaintext hoặc ciphertext tùy theo trạng thái của FernetCipher. Trường encrypted được dùng để peer nhận biết có cần giải mã trước khi lưu hay không. Nếu giải mã thất bại, peer nhận trả lỗi thay vì lưu nội dung không hợp lệ.

ACK là cơ chế xác nhận ở tầng ứng dụng. Sau khi peer nhận xử lý một thông điệp hợp lệ, nó trả ACK có trường `ack_for` trỏ đến `message_id` của thông điệp gốc. Peer gửi chỉ xem việc gửi thành công khi nhận được ACK đúng. Nếu kết nối lỗi hoặc phản hồi không đúng, hệ thống xem đó là lỗi delivery. Nếu tin đã gửi đi nhưng ACK không về kịp thời, hệ thống phân biệt thành trạng thái `sent_no_ack`, nghĩa là sender không chắc chắn hoàn toàn về kết quả ở phía nhận.

6. Peer discovery



Hình 6.1: Quy trình peer discovery thông qua bootstrap sever

Peer discovery là cơ chế giúp một peer biết sự tồn tại và địa chỉ của peer khác. Trong hệ thống này, Bootstrap Server đóng vai trò tracker. Khi một peer đăng ký, Bootstrap ghi nhận username, host và port của peer đó, đồng thời trả về danh sách peer khác. Peer cũng có thể chủ động refresh danh sách bằng yêu cầu peer list.

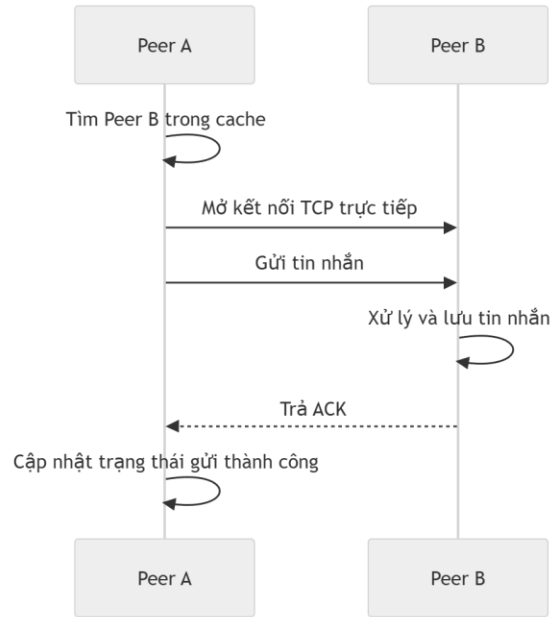
Danh sách peer sau khi nhận được được lưu trong cache cục bộ của peer. Cache này nằm trong bộ nhớ của PeerNode, không phải file bền vững. Nhờ cache, peer có thể tiếp tục sử dụng địa chỉ đã biết để kết nối trực tiếp đến peer khác ngay cả khi Bootstrap tạm thời không phản hồi. Điều này chỉ đúng trong phạm vi peer đã được khám phá trước đó và địa chỉ TCP vẫn còn reachable. Nếu peer đích đổi port, restart với trạng thái mới, hoặc chưa từng được discovery, peer gửi vẫn cần Bootstrap để lấy thông tin mới.

Bootstrap cũng quản lý trạng thái peer. Khi peer gửi heartbeat đều đặn, trạng thái được xem là online. Khi peer gọi disconnect, trạng thái chuyển thành offline. Khi heartbeat bị mất quá ngưỡng, trạng thái chuyển thành disconnected. Các trạng thái này hỗ trợ UI và quyết định gửi offline queue, nhưng không phải một đảm bảo tuyệt đối về khả năng kết nối, vì trạng thái mạng thực tế có thể thay đổi giữa hai lần cập nhật.

Một đặc điểm quan trọng trong implementation là port reservation. Trong cùng một phiên Bootstrap, nếu một port đã được gán với một username, peer khác không thể đăng ký chiếm port đó. Cùng username cũng phải tái sử dụng port đã reserve. Cơ chế này giúp tránh tình trạng hai peer có định danh hoặc endpoint mâu thuẫn trong cùng phiên chạy.

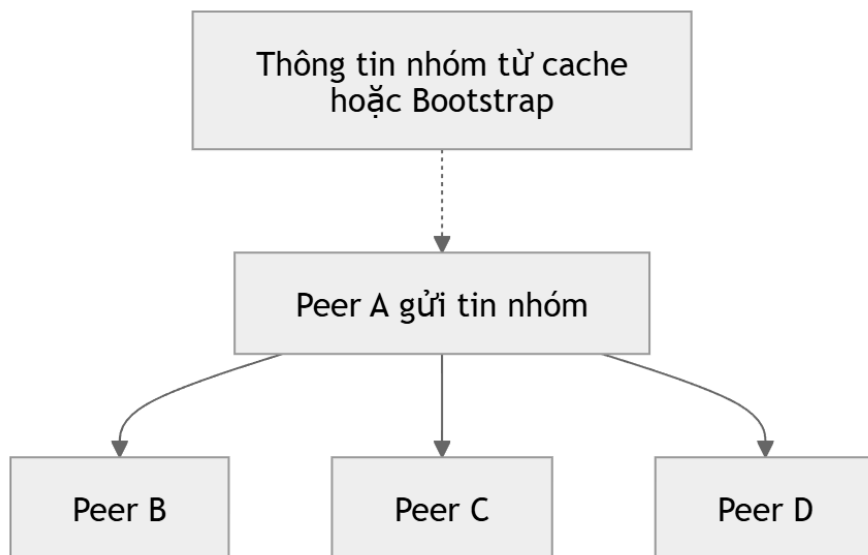
7. Chat trực tiếp, chat nhóm và broadcast

Chat trực tiếp là luồng đơn giản nhất. Người gửi chọn peer đích, nhập nội dung và gửi qua Peer Web UI hoặc CLI. Peer gửi tìm endpoint của peer đích, mã hóa nội dung nếu cơ chế Fernet đang bật, tạo thông điệp CHAT, mở TCP connection đến peer nhận và chờ ACK. Peer nhận giải mã nếu cần, lưu thông điệp vào lịch sử chat cục bộ và thông báo lên Web UI nếu có callback runtime.



Hình 7.1: Luồng gửi tin nhắn trực tiếp giữa hai peer.

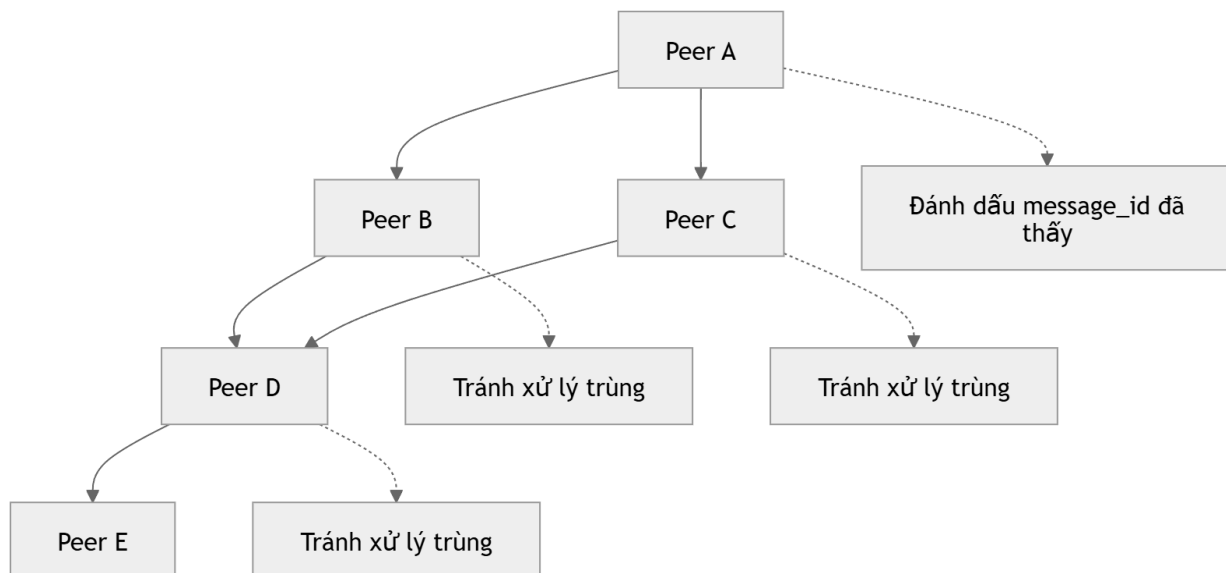
Chat nhóm trong hệ thống hiện tại được triển khai theo hướng persistent group metadata. Người dùng có thể tạo nhóm, tham gia nhóm, rời nhóm và trong trường hợp owner có thể thêm hoặc xóa thành viên. Metadata nhóm được lưu trong Bootstrap Registry trong phiên chạy. Khi một thành viên gửi tin nhắn nhóm, sender lấy danh sách thành viên của nhóm và gửi trực tiếp đến từng thành viên còn lại. Nội dung tin nhắn nhóm không đi qua Bootstrap.



Hình 7.2: Cơ chế gửi tin nhắn nhóm bằng cách gửi trực tiếp đến từng thành viên.

Khi một thành viên rời nhóm, Bootstrap cập nhật danh sách thành viên. Peer rời nhóm cũng gửi thông báo hệ thống đến các thành viên còn lại nếu có thể, để UI của các peer khác hiển thị sự kiện rời nhóm. Chức năng này đã được kiểm thử trong Web API integration: Bob join nhóm do Alice tạo, gửi tin nhắn nhóm, rời nhóm và Alice nhận thông báo Bob rời nhóm.

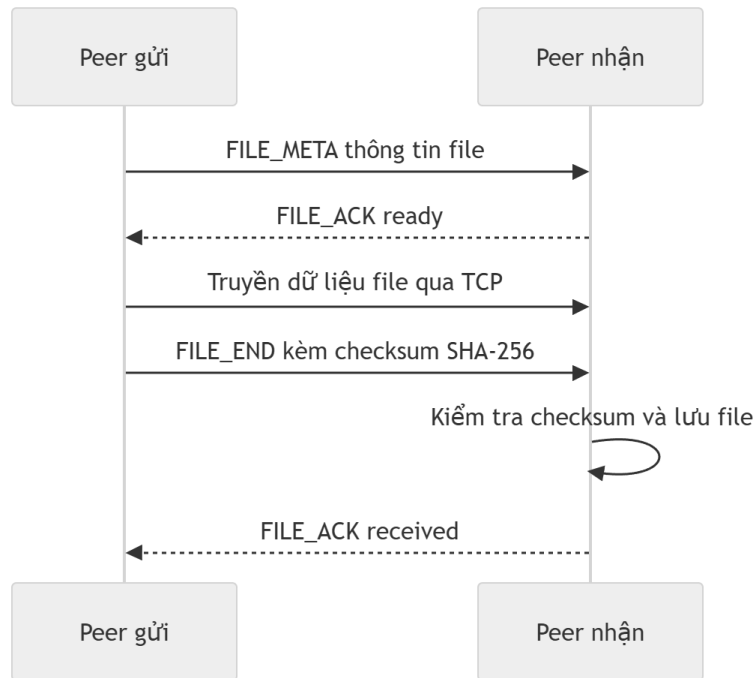
Broadcast được dùng khi một peer muốn gửi một thông điệp đến các peer đã biết. Thay vì gọi Bootstrap để chuyển tiếp, peer gửi tạo một thông điệp broadcast và gửi đến các peer online trong cache. Peer nhận có thể tiếp tục forward để lan truyền thông điệp trong mạng đã biết. Để tránh vòng lặp đơn giản, mỗi handler lưu tập `seen_message_ids`; nếu đã xử lý một broadcast có cùng message id, peer chỉ trả ACK và không forward lại. Cơ chế này phù hợp với phạm vi đề án nhưng chưa phải thuật toán broadcast tối ưu cho mạng lớn.



Hình 7.3: Cơ chế broadcast lan truyền trong mạng peer đã biết.

8. Truyền file giữa các peer

Truyền file đã được triển khai trong hệ thống và dùng kênh TCP trực tiếp giữa các peer. Bootstrap không relay nội dung file và cũng không lưu file offline. Người gửi có thể gửi file trực tiếp cho một peer, gửi file đến các thành viên trong nhóm hoặc gửi file broadcast đến các peer đã biết. Trong trường hợp gửi theo nhóm hoặc broadcast, sender thực hiện nhiều lần truyền trực tiếp đến từng peer đích, tương tự cơ chế fan-out của group chat.



Hình 8.1: Quy trình truyền file trực tiếp giữa hai peer.

Luồng truyền file bắt đầu bằng metadata. Sender kiểm tra file tồn tại, không rỗng và không vượt quá giới hạn kích thước cấu hình là 50 MB. Sau đó sender gửi metadata gồm transfer id, tên file, kích thước, MIME type, mode gửi và thông tin nhóm nếu có. Receiver trả trạng thái sẵn sàng, rồi sender truyền raw bytes của file qua cùng socket. Khi toàn bộ bytes đã được gửi, sender gửi checksum SHA-256. Receiver tự tính checksum từ dữ liệu đã nhận; nếu khớp, file được xem là nhận thành công, receiver trả ACK trạng thái received và lưu file vào storage/received_files/<username>/.

Hệ thống có xử lý trùng tên file khi lưu. Nếu receiver đã có file cùng tên, tên mới được thêm hậu tố số như sample_1.txt để tránh ghi đè. Web UI hiển thị file đã nhận và cung cấp đường dẫn để mở hoặc tải xuống file. Phần download/open có kiểm tra filename để hạn chế path traversal: filename không được là đường dẫn tuyệt đối, không được chứa path component không hợp lệ và phải nằm trong danh sách file đã nhận của phiên peer.

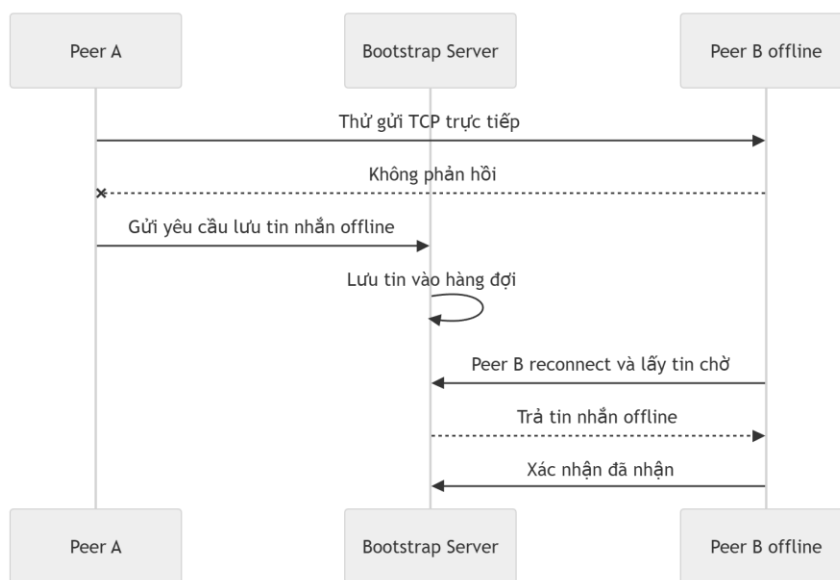
Giới hạn quan trọng là file transfer yêu cầu peer đích online và reachable tại thời điểm gửi. Hệ thống không có offline file delivery. Nếu peer đích offline, chưa có trong cache, hoặc TCP connection thất bại, file không được đưa vào offline queue. Offline queue hiện chỉ áp dụng cho tin nhắn văn bản.

9. Xử lý peer offline và truyền tin đáng tin cậy

Độ tin cậy của truyền tin trong hệ thống được xây dựng ở mức ứng dụng thông qua ACK, timeout và hàng đợi offline cho tin nhắn văn bản. Khi gửi thông điệp trực tiếp, PeerClient không chỉ gửi dữ liệu mà còn chờ ACK. ACK phải có loại ACK và phải xác nhận đúng message_id. Nếu không đạt điều kiện này, sender xem quá trình gửi không thành công.

Khi peer đích không reachable, hệ thống phân biệt một số trường hợp. Nếu không kết nối được hoặc protocol lỗi, DeliveryError được sinh ra. Nếu dữ liệu có thể đã gửi nhưng ACK không về đúng hạn, AckTimeoutError được sinh ra và tầng trên có thể ghi nhận trạng thái sent_no_ack. Sự phân biệt này phản ánh một vấn đề thực tế của hệ thống phân tán: sender có thể không luôn biết chắc receiver đã xử lý thông điệp hay chưa khi phản hồi bị mất.

Offline queue được triển khai cho tin nhắn văn bản. Nếu peer đích offline hoặc gửi thất bại trong khi Bootstrap còn online, sender có thể yêu cầu Bootstrap lưu bản ghi offline. Khi peer đích khởi động hoặc gọi fetch pending messages, nó lấy các tin đang chờ, đưa vào handler để lưu như tin đã nhận, sau đó đánh dấu delivered với Bootstrap. Nếu Bootstrap không khả dụng, peer gửi có cơ chế fallback ghi hàng đợi offline cục bộ vào file dưới storage/offline_messages/<peer_id>.json. Vòng retry định kỳ của peer sẽ thử refresh peer, fetch pending messages và gửi lại các tin trong hàng đợi cục bộ.



Hình 9.1: Cơ chế lưu và nhận lại tin nhắn khi peer đích offline.

Cần nhấn mạnh rằng offline queue không biến Bootstrap thành relay cho tin nhắn online. Nó chỉ lưu nội dung text khi gửi trực tiếp không thể thực hiện. Ngoài ra, offline queue của Bootstrap hiện tồn tại trong bộ nhớ của phiên chạy, nên nếu Bootstrap restart thì các tin chưa giao có thể mất. File transfer không tham gia cơ chế offline queue.

Khi Bootstrap offline, khả năng của hệ thống bị giới hạn nhưng không mất hoàn toàn. Peer không thể discovery peer mới, không thể đồng bộ group mới, không thể ghi offline queue lên Bootstrap và Admin UI không thể lấy dữ liệu runtime. Tuy nhiên, các peer đã có cache địa chỉ nhau vẫn có thể tiếp tục gửi tin trực tiếp, gửi group message dựa trên group cache, broadcast đến peer đã biết và truyền file trực tiếp nếu endpoint TCP còn hoạt động.

10. Xử lý lỗi và tính chịu lỗi

Hệ thống xử lý lỗi ở nhiều lớp. Ở lớp protocol, thông điệp phải có loại hợp lệ, message id và timestamp. Nếu frame JSON sai, thiếu trường bắt buộc hoặc vượt quá kích thước tối đa, hệ thống sinh lỗi protocol và trả ERROR khi có thể. Điều này giúp tránh việc handler xử lý dữ liệu không hợp lệ như một thông điệp bình thường.

Ở lớp Bootstrap Registry, hệ thống kiểm tra trùng username, trùng port và quyền thao tác nhóm. Peer khác không được chiếm port đã reserve trong phiên Bootstrap. Cùng username phải tái sử dụng port đã được gán. Với nhóm, chỉ owner có quyền thêm hoặc xóa thành viên bằng các thao tác owner-controlled. Peer không phải thành viên không được gửi tin trong managed group vì send_managed_group kiểm tra membership trước khi fan-out.

Khi peer disconnect đúng cách, Bootstrap đánh dấu peer là offline thay vì xóa record. Điều này cho phép peer reconnect bằng đúng port trong cùng phiên Bootstrap. Khi peer biến mất không báo trước, heartbeat timeout giúp Bootstrap đánh dấu disconnected. Trạng thái này hỗ trợ UI và quyết định queue, nhưng không thay thế kiểm tra kết nối TCP thực tế khi gửi.

Với file transfer, hệ thống kiểm tra file đầu vào, giới hạn kích thước, trạng thái online của target, metadata hợp lệ, kết nối bị đóng sớm, thiếu thông điệp kết thúc và checksum mismatch. Nếu checksum không khớp, receiver xóa file đã ghi để tránh lưu file hỏng. Đây là cơ chế quan trọng vì truyền file qua TCP stream cần đảm bảo file nhận được đúng với file gửi.

Khi Bootstrap offline, các thao tác phụ thuộc Bootstrap sẽ trả cảnh báo hoặc lỗi. Peer Web UI có thể hiển thị trạng thái bootstrap offline khi refresh peers hoặc groups thất bại. Admin UI cũng hiển thị offline nếu không proxy được dữ liệu từ Bootstrap. Trong tình huống này, hệ thống vẫn cố gắng dùng cache cục bộ cho các chức năng có thể thực hiện trực tiếp, nhưng không phóng đại thành khả năng hoạt động độc lập hoàn toàn.

11. Kiểm thử và demo hệ thống

11.1 Kiểm thử

Các kiểm thử tự động trong dự án tập trung vào hành vi hệ thống thay vì chỉ kiểm tra từng hàm riêng lẻ. Bộ test bao phủ direct socket, ACK, group fan-out, registry, offline queue, mã hóa text, broadcast duplicate, file transfer streaming, log Web UI, group join/leave và một số lỗi đầu vào. Trong môi trường đã có dependencies, bộ test hiện có 18 testcase.

Tác nhân	Mục tiêu	Cách thực hiện	Kết quả mong đợi	Kết quả đạt được
Direct chat qua TCP	Xác nhận peer nhận CHAT, lưu lịch sử và trả ACK đúng message id	Khởi động PeerServer của Bob trên port tự do, Alice dùng PeerClient gửi tin	Bob trả ACK, ack_for trùng message id, file history được tạo	Đạt
Peer không reachable	Kiểm tra lỗi khi gửi đến endpoint không có peer lắng nghe	Gửi tin đến 127.0.0.1:9 với timeout ngắn	Sinh DeliveryError	Đạt
Group chat fan-out	Kiểm tra gửi nhóm gọi gửi trực tiếp đến từng member	Dùng fake peer và gửi đến Bob, Carol	Mỗi target được gọi với group=True	Đạt
Leave group	Kiểm tra thành viên rời nhóm không còn thấy nhóm trong danh sách của mình	Alice tạo group có Bob, Bob leave group	Bob không còn group trong list_groups, group còn Alice	Đạt
Offline queue Bootstrap	Kiểm tra hàng đợi tin nhắn offline trong registry	Bob offline, Alice queue tin, Bob fetch pending và mark delivered	Pending có tin chờ, sau mark delivered pending rỗng	Đạt

Offline queue cục bộ	Kiểm tra fallback khi Bootstrap queue không khả dụng	Tạo PeerNode không start Bootstrap, gửi đến target không tồn tại	File offline queue cục bộ chứa target và message	Đạt
Bootstrap shutdown sau discovery	Kiểm tra khả năng tiếp tục dùng cache khi Bootstrap không có mặt	Sau khi peer đã biết nhau, thao tác trực tiếp có thể dùng endpoint cache	Chat/file trực tiếp vẫn có thể hoạt động nếu TCP target còn reachable; discovery mới không khả dụng	Tạm ổn
Reconnect peer	Kiểm tra peer offline reconnect cùng port trong phiên Bootstrap	Register Bob, mark offline, register lại Bob cùng port	Status chuyển lại online, port giữ nguyên	Đạt
Duplicate username/port	Kiểm tra ràng buộc identity và endpoint	Thử đăng ký port đã reserve hoặc username đổi port	Registry từ chối bằng ValueError	Đạt
File transfer trực tiếp	Kiểm tra streaming file qua TCP và lưu file nhận	Bob mở PeerServer, Alice cache Bob và gửi cùng file hai lần	Bob nhận đủ nội dung, file trùng tên được đổi tên an toàn	Đạt
Group trong Web UI	Kiểm tra tạo group, join, gửi tin, leave và thông báo hệ thống	Alice và Bob đăng ký qua Flask test client, Bob join group của Alice rồi gửi và rời nhóm	Log nhóm đúng, Bob rời nhóm, Alice thấy thông báo leave	Đạt

Broadcast trong Web UI	Kiểm tra gửi broadcast text qua API UI	Peer gửi thông điệp broadcast đến peer đã biết	Các peer reachable nhận log broadcast	Đạt
File transfer group/broadcast	Kiểm tra gửi file đến nhiều peer	Sender chọn group hoặc broadcast file trong UI	Peer online nhận file trực tiếp, peer offline bị skipped	Đạt
Admin UI refresh	Kiểm tra Admin UI quan sát Bootstrap	Admin UI gọi proxy đến Bootstrap stats, peers, groups, offline messages	Khi Bootstrap online có dữ liệu, khi offline hiển thị unavailable	Đạt
Log Web UI	Đảm bảo log runtime chỉ chứa thông tin người dùng cần thấy	Thêm log CHAT, SYSTEM và các loại debug/register/heartbeat	Log bỏ các loại không liên quan, clear state xóa log	Đạt
Mã hóa text	Kiểm tra nội dung encrypted được decrypt trước khi lưu	Tạo Fernet key, gửi CHAT có encrypted=True	History chứa plaintext, không chứa ciphertext	Đạt

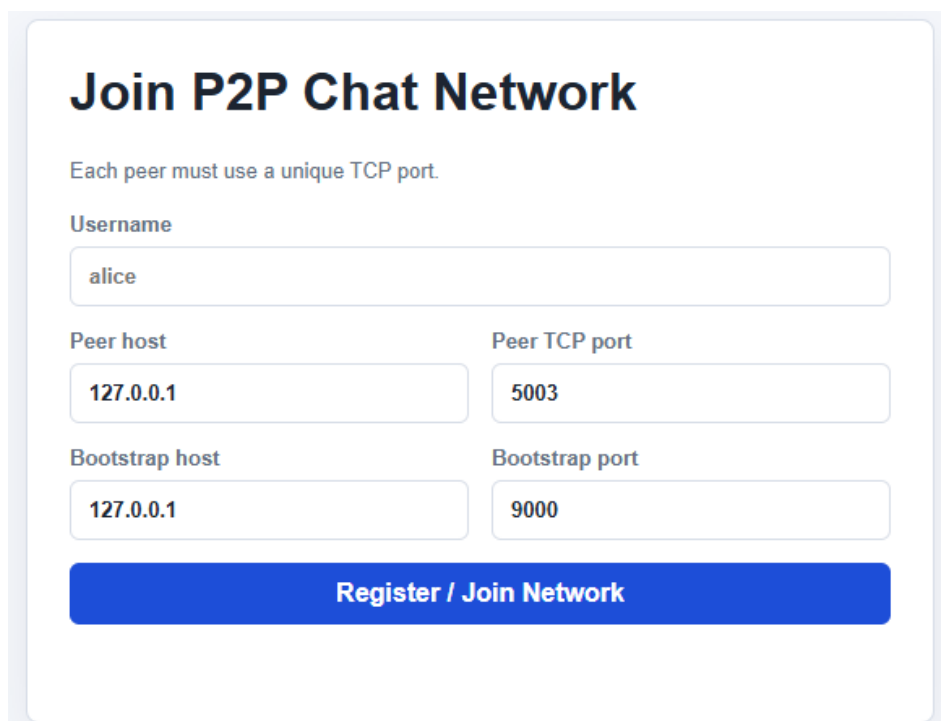
Kết quả kiểm thử cho thấy các chức năng cốt lõi của đồ án đã hoạt động trong phạm vi local test. Tuy nhiên, vẫn còn một số tình huống nên được bổ sung test tự động nếu mở rộng dự án, chẳng hạn kiểm thử group file và broadcast file với nhiều peer thật, kiểm thử Admin UI proxy khi Bootstrap offline, kiểm thử restart Bootstrap làm mất offline queue runtime và kiểm thử tải với nhiều peer đồng thời.

Với môi trường có pytest và dependencies đã cài, có thể chạy python -m pytest.

11.2 Demo hệ thống

a, Đăng ký peer mới

Để truy cập vào hệ thống chat P2P người dùng bắt buộc phải đăng ký tên và port với bootstrap sever để hệ thống nhận diện rằng đã có 1 peer mới đăng ký



Join P2P Chat Network

Each peer must use a unique TCP port.

Username
alice

Peer host
127.0.0.1

Peer TCP port
5003

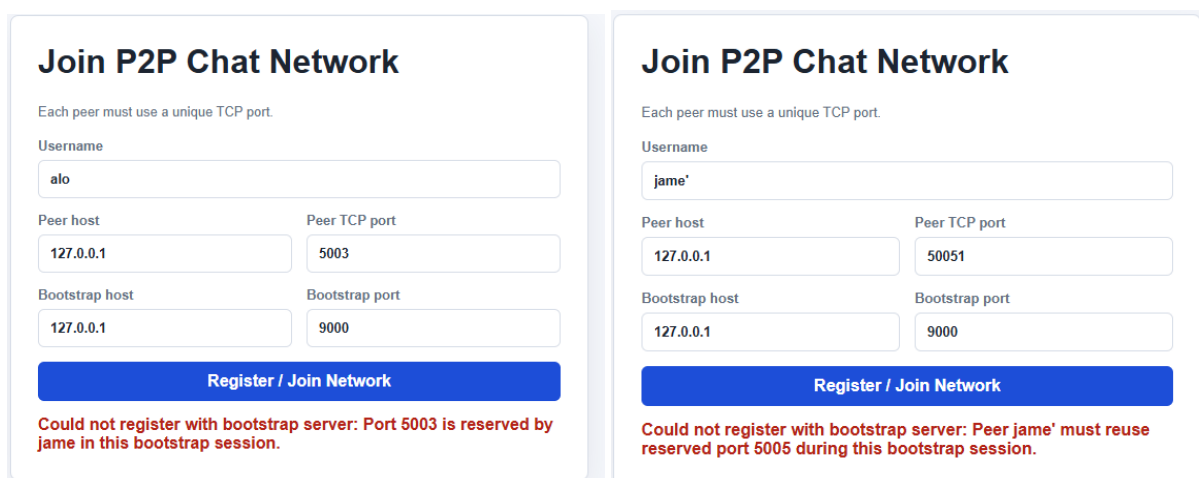
Bootstrap host
127.0.0.1

Bootstrap port
9000

Register / Join Network

Hình 11.1 Giao diện đăng ký peer mới.

Đối với trường hợp người dùng đăng ký tên hoặc cổng đã trùng với một peer đã có trong hệ thống thì hệ thống sẽ báo lỗi phía dưới.



Join P2P Chat Network

Each peer must use a unique TCP port.

Username
alo

Peer host
127.0.0.1

Peer TCP port
5003

Bootstrap host
127.0.0.1

Bootstrap port
9000

Register / Join Network

Could not register with bootstrap server: Port 5003 is reserved by jame in this bootstrap session.

Join P2P Chat Network

Each peer must use a unique TCP port.

Username
jame'

Peer host
127.0.0.1

Peer TCP port
50051

Bootstrap host
127.0.0.1

Bootstrap port
9000

Register / Join Network

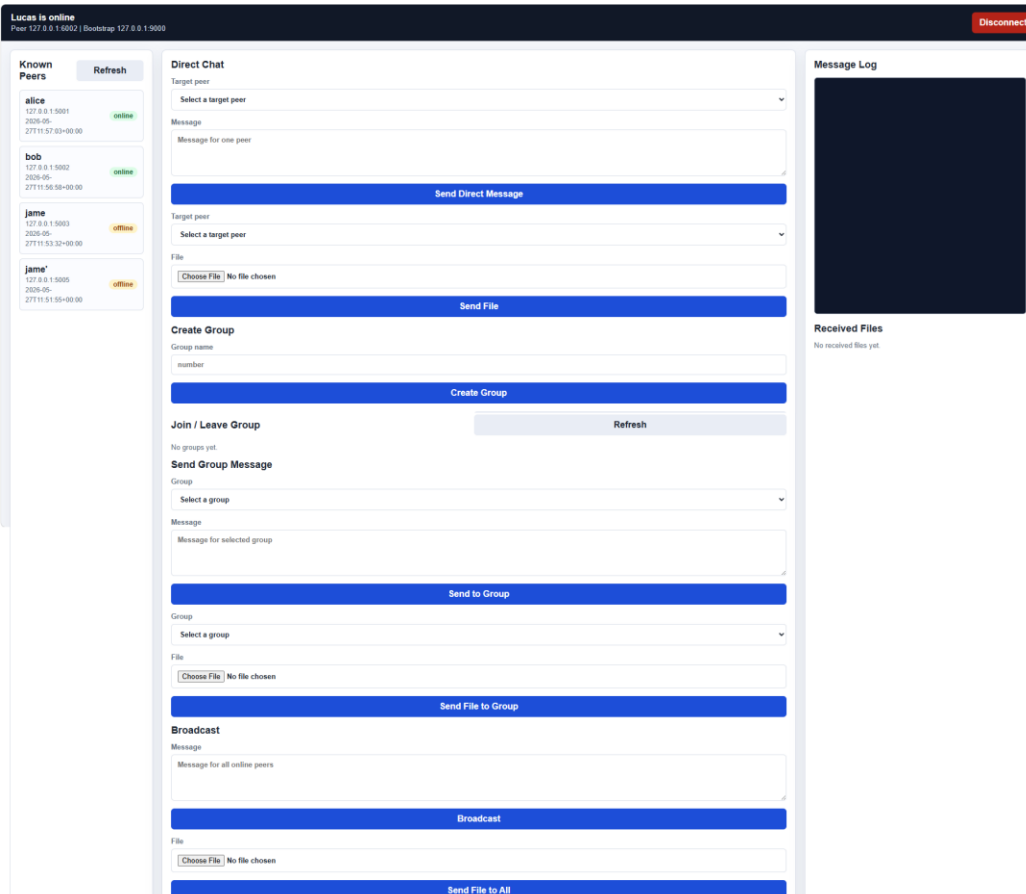
Could not register with bootstrap server: Peer jame' must reuse reserved port 5005 during this bootstrap session.

Hình 11.2 Hệ thống thông báo lỗi khi peer mới trùng tên hoặc port của peer cũ.

b, Giao diện chính của hệ thống

Hệ thống bao gồm 3 cách giao tiếp chính:

- Giao tiếp giữa peer A và peer B (gửi tin nhắn trực tiếp)
- Giao tiếp nhóm gồm nhiều peer với nhau
- Giao tiếp broadcast (giao tiếp toàn hệ thống).

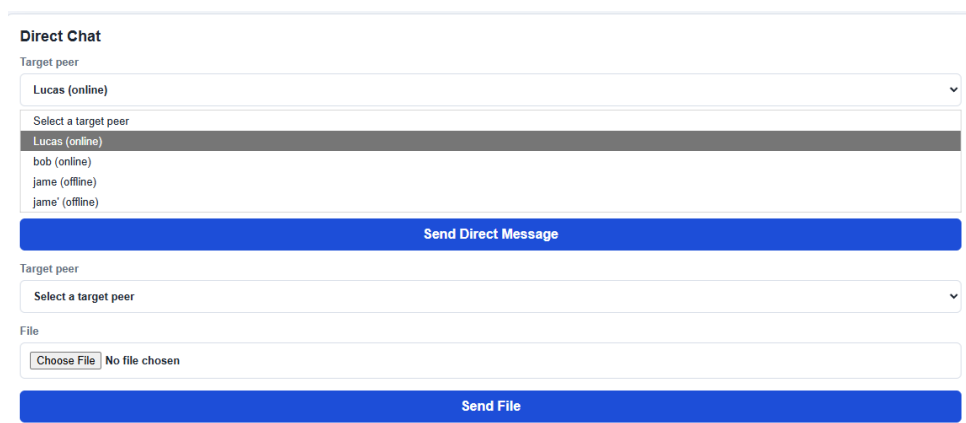


Hình 11.3: Giao diện trang chủ của hệ thống chat P2P.

Mô tả hệ thống:

- Phía trên bên cùng là hiển thị thông tin của peer đang sở hữu gồm: tên, địa chỉ ip của peer và bootstrap sever.
- Góc trên bên phải là nút đăng xuất (giúp peer có thể đăng xuất tạm thời khỏi phiên chat).
- Ngay phía dưới là: các thông tin các peer khác mà peer đang sở hữu biết. Bao gồm các peer online và offline.
- Bên phải là giao diện theo dõi và nhận tin nhắn từ các peer khác.

1. Giao tiếp giữa các peer (gửi tin nhắn trực tiếp):



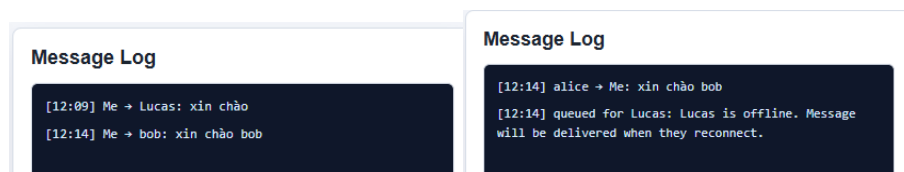
Hình 11.4: Giao diện gửi tin nhắn trực tiếp.

Người dùng thao tác với chức năng Direct chat để sử dụng chức năng gửi tin nhắn trực tiếp. Khi chọn 1 peer bất kỳ để gửi tin nhắn người dùng click vào optional box để chọn, hệ thống hiển thị đầy đủ các peer hiện có (cả online lẫn offline).

Người dùng có thể chọn gửi tin nhắn hoặc gửi file cho peer khác ngay ở dưới, sau khi hoàn thành nhập nội dung người gửi bấm send để gửi.

Ví dụ: Peer alice gửi tin nhắn xin chào cho cả bob(online) và lucas (offline)

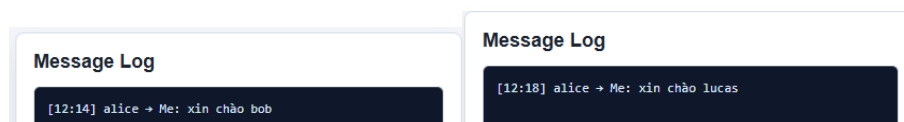
Đối với peer bob(online) hệ thống thông báo đã gửi thành công, và với peer lucas(offline) hệ thống thông báo peer lucas đang offline và sẽ gửi lại khi online.



Hình 11.5: Message Log của alice gửi cho bob (bên trái) và gửi cho lucas(bên phải).

Kết quả:

Đối với peer bob(online) tin nhắn sẽ được truyền đến luôn. Còn peer lucas(offline) sẽ nhận được tin nhắn sau khi login lại.



Hình 11.6: Message Log bob (bên trái) và lucas(bên phải) .

2. Giao tiếp thông qua group

Người dùng trực tiếp thao tác với chức năng create group để tạo nhóm.

Với bước đầu là đặt tên (VD: tạo nhóm tên Big city), nhóm sẽ được tạo public gồm tên chủ sở hữu, tên các member, số lượng thành viên, và trạng thái truy cập của peer.

Đối với peer trong nhóm có thể rời và gửi tin nhắn trong nhóm

The screenshot shows a web interface for creating and managing a group. At the top, there's a 'Create Group' section with a 'Group name' input field containing 'Big city' and a blue 'Create Group' button. Below this is a 'Join / Leave Group' section with a 'Refresh' button. The main part of the interface is the 'Send Group Message' section. It features a 'Group' dropdown menu currently showing 'Big city (1)', a 'Message' input field with the placeholder 'Message for selected group', and a blue 'Send to Group' button. At the bottom, there's a file upload section with a 'Select a group' dropdown, a 'File' input field with a 'Choose File' button and 'No file chosen' text, and a blue 'Send File to Group' button.

Hình 11.7: Giao diện gửi tin nhắn qua group.

Đối với peer chưa gia nhập khi sử dụng chức năng này sẽ thấy luôn các nhóm đang tồn tại cùng button join nhóm.

This screenshot shows a portion of the web interface, specifically the 'Join / Leave Group' section. It includes a 'Refresh' button and a group entry for 'Big city'. The entry shows 'Owner: bob | Members: bob | Count: 1 | Status: not joined' and a blue 'Join Group' button.

Hình 11.8: Giao diện group đối với các peer mới.

Ví dụ:

Lấy peer alice gửi tin nhắn đến nhóm

```
[12:27] Me → Group(Big city): Xin chào mọi người nhé
```

Hình 11.9: Peer alice gửi tin nhắn đến group Big city.

Các peer khác sẽ thấy trong Message Log tin nhắn của alice gửi đến:

```
[12:27] alice → Group(Big city): Xin chào mọi người  
nhé
```

Hình 11.10: Message log của các peer trong group Big city.

Khi có thành viên rời nhóm message log sẽ hiển thị thông báo tới toàn mọi người trong nhóm rằng peer đó đã rời nhóm

```
[12:30] Lucas left Group(Big city)
```

Hình 11.11: Hệ thống thông báo đã có người rời nhóm trong message log.

3. Giao tiếp gửi tin nhắn qua broadcast

Người dùng có thể gửi tin nhắn đến tất cả mọi người thông qua chức năng này.

Broadcast

Message

Message for all online peers

Broadcast

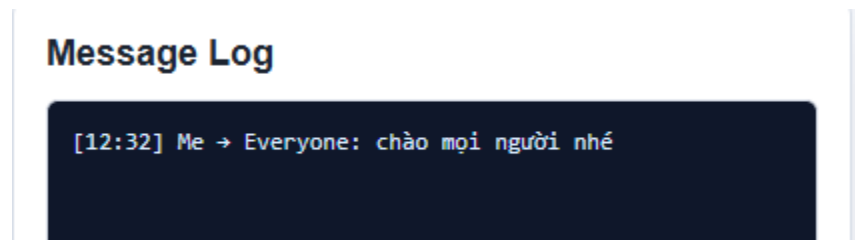
File

Choose File No file chosen

Send File to All

Hình 11.12: Giao diện gửi tin nhắn qua broadcast.

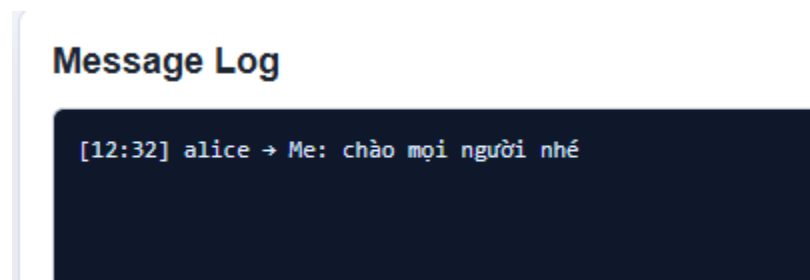
Ví dụ: peer alice gửi tin nhắn broadcast:



Hình 11.13: Peer alice gửi tin nhắn thông qua broadcast.

Kết quả:

Peer bob và Lucas đều nhận được



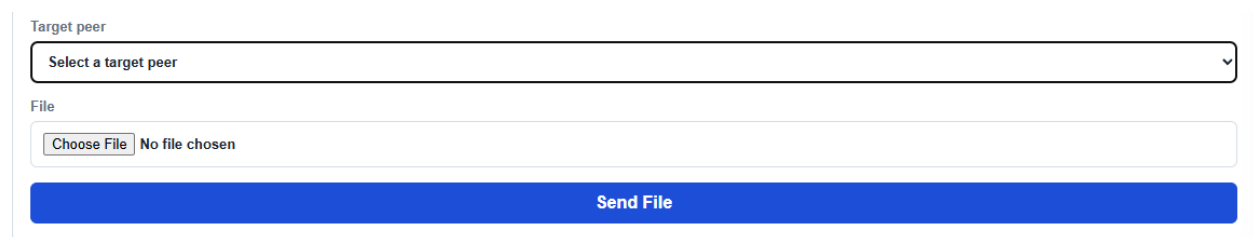
Hình 11.14: Các peer trong hệ thống đồng loạt nhận được tin nhắn.

4. Chức năng gửi file (mở rộng)

Đối với từng loại gửi tin nhắn đều có thêm chức năng gửi file.

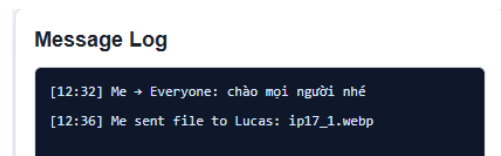
- Gửi trực tiếp từ peer này qua peer khác:

Người dùng phải chọn peer rồi chọn file muốn gửi rồi gửi.



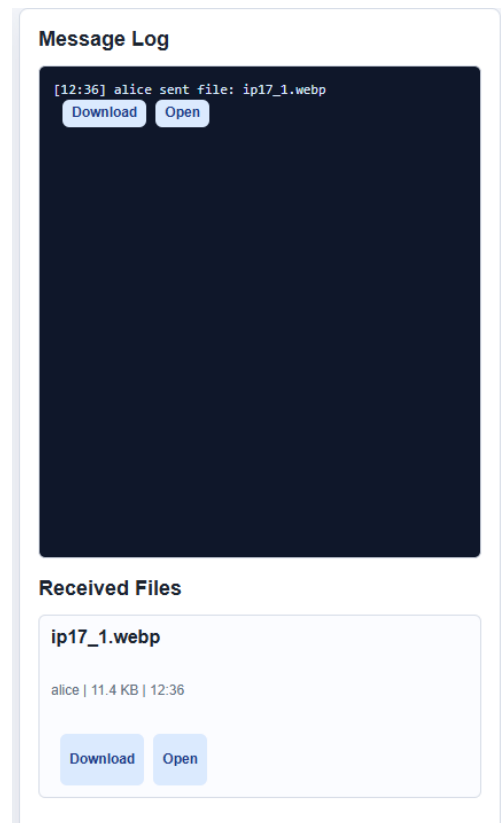
Hình 11.15: Giao diện gửi file qua tin nhắn trực tiếp.

Ví dụ: peer alice gửi ảnh qua cho peer lucas



Hình 11.16: Message log thông báo peer alice đã gửi file cho lucas.

Phía peer lucas sẽ nhận được thông báo và có thể chọn download về máy hoặc mở trực tiếp, ngay phía bên dưới là tổng hợp tất cả các file đã nhận được.



Hình 11.17: Giao diện của peer lucas khi nhận file.

**** Gửi file với group và broadcast cũng tương tự.***

12. Đánh giá

12.1 Ưu điểm

Hệ thống thể hiện rõ mô hình hybrid P2P. Bootstrap Server hỗ trợ discovery và metadata, nhưng tin nhắn online và file đi qua TCP trực tiếp giữa peer. Điều này giúp đồ án minh họa được bản chất của peer node vừa là client vừa là server.

Thiết kế module tương đối rõ ràng. Protocol, peer client, peer server, message handler, discovery, file transfer, registry và web UI được tách thành các thành phần riêng. Cách tách này giúp kiểm thử từng phần thuận lợi hơn và làm rõ trách nhiệm của từng lớp.

Cơ chế ACK và timeout giúp hệ thống không gửi tin theo kiểu fire-and-forget hoàn toàn. Sender có thể biết khi nào receiver đã xác nhận message, khi nào message có thể đã gửi nhưng chưa có ACK và khi nào kết nối thất bại. Đây là yếu tố quan trọng khi giải thích độ tin cậy trong hệ thống phân tán.

Chức năng nhóm và broadcast không phụ thuộc Bootstrap như một relay nội dung. Bootstrap chỉ quản lý danh sách thành viên và group metadata, còn sender tự fan-out đến các member. Cách làm này phù hợp với định hướng P2P của đồ án.

Truyền file được triển khai bằng streaming raw bytes, có kiểm tra kích thước, checksum SHA-256, xử lý trùng tên file và lưu file nhận theo từng peer. Đây là một chức năng thực tế, vượt ra ngoài chat văn bản cơ bản.

Hệ thống có kiểm thử tự động cho nhiều hành vi chính, bao gồm TCP direct message, offline queue, group, file transfer, web logs và encryption. Điều này làm tăng độ tin cậy của báo cáo vì các mô tả không chỉ dựa trên ý tưởng mà có đối chiếu với mã nguồn và test.

12.2 Hạn chế

Bootstrap Server vẫn là điểm phụ thuộc quan trọng cho discovery, group metadata và offline queue. Khi Bootstrap offline, hệ thống chỉ tiếp tục hoạt động trong phạm vi thông tin đã cache. Peer mới không thể tham gia mạng theo cách bình thường và peer hiện tại không thể lấy danh sách peer mới.

Trạng thái Bootstrap không được lưu bền vững bằng database. Registry, groups và offline queue chủ yếu nằm trong bộ nhớ của phiên chạy. Vì vậy hệ thống chưa phù hợp để triển khai dài hạn nếu cần khôi phục đầy đủ sau khi server restart.

Cache của peer cũng chủ yếu nằm trong bộ nhớ. Sau khi peer process restart, nó cần Bootstrap để discovery lại. Điều này giới hạn khả năng hoạt động độc lập nếu Bootstrap không sẵn sàng.

Mã hóa hiện chỉ là cơ chế shared-key Fernet cho text payload, chưa phải end-to-end encryption hoàn chỉnh. Hệ thống chưa có xác thực peer, chưa có kiểm soát quyền dựa trên danh tính mạnh và chưa bảo vệ đầy đủ trước peer giả mạo trong cùng môi trường mạng.

File transfer không hỗ trợ offline delivery, không hỗ trợ resume khi bị gián đoạn và chưa mã hóa nội dung file ở tầng ứng dụng. Nếu peer đích offline hoặc kết nối bị lỗi, file phải được gửi lại thủ công.

Broadcast dùng flooding đơn giản và dựa vào seen_message_ids để tránh lặp. Cách này dễ hiểu trong phạm vi đồ án nhưng chưa tối ưu cho mạng lớn, chưa có TTL, chưa có topology-aware routing và chưa đánh giá tải khi số peer tăng.

Giao diện Web UI và Admin UI được viết inline trong file Python bằng chuỗi HTML/CSS/JavaScript. Cách này phù hợp cho đồ án nhỏ và demo nhanh, nhưng sẽ khó bảo trì nếu giao diện tiếp tục mở rộng.

12.3 Hướng phát triển

Một hướng phát triển quan trọng là bổ sung persistence bằng SQLite hoặc PostgreSQL cho Bootstrap Server. Khi đó peer registry, group metadata và offline queue có thể tồn tại qua restart, giúp hệ thống ổn định hơn trong vận hành.

Hệ thống cũng có thể được mở rộng bằng cơ chế xác thực peer. Username hiện chỉ là định danh đơn giản. Nếu bổ sung token, mật khẩu hoặc public-key identity, Bootstrap có thể hạn chế giả mạo peer và kiểm soát tốt hơn các thao tác group.

Về bảo mật, có thể thay cơ chế shared-key bằng key exchange theo peer hoặc theo nhóm, từ đó triển khai end-to-end encryption đúng nghĩa cho từng cuộc hội thoại. File transfer cũng cần được mã hóa ở tầng ứng dụng nếu sử dụng trong môi trường không tin cậy.

Đối với P2P qua Internet, cần nghiên cứu NAT traversal như STUN/TURN hoặc chuyển một phần data channel sang WebRTC.

Về truyền file, hệ thống có thể bổ sung resume transfer, checksum từng chunk, tiến độ truyền và retry từng phần thay vì gửi lại toàn bộ file. Với broadcast, có thể

bổ sung TTL, giới hạn hop hoặc thuật toán định tuyến theo topology để giảm số kết nối dư thừa.

Cuối cùng, giao diện có thể được tách thành template và static assets riêng, hoặc chuyển thành frontend app độc lập. Việc này giúp mã nguồn dễ bảo trì hơn, nhất là khi bổ sung danh sách hội thoại, trạng thái gửi, tiến độ file và thông báo lỗi chi tiết.

13. Kết luận

P2P Chat System đã hiện thực một hệ thống chat ngang hàng theo mô hình hybrid P2P trong phạm vi local hoặc LAN. Bootstrap Server đóng vai trò hỗ trợ khám phá peer, quản lý trạng thái, metadata nhóm và hàng đợi tin nhắn văn bản offline, nhưng không relay tin nhắn online và không relay nội dung file. Sau khi peer đã biết địa chỉ của nhau, các thao tác chat trực tiếp, chat nhóm, broadcast và truyền file được thực hiện qua TCP socket trực tiếp giữa các peer.

Hệ thống có các thành phần cơ bản của một đồ án hệ thống phân tán: peer vừa client vừa server, bootstrap tracker, heartbeat, ACK, timeout, cache cục bộ, offline queue, group membership, broadcast flooding, file streaming và giao diện quan sát. Kết quả kiểm thử cho thấy các chức năng cốt lõi hoạt động đúng trong phạm vi đã triển khai.

Mặc dù vậy, hệ thống vẫn còn các giới hạn rõ ràng về persistence, bảo mật, NAT traversal, khả năng mở rộng broadcast và độ bền của offline queue. Những giới hạn này không làm giảm giá trị của đồ án trong việc minh họa kiến trúc P2P, nhưng cần được nêu rõ để tránh hiểu nhầm hệ thống như một sản phẩm chat Internet hoàn chỉnh. Hiện tại hệ thống mới chủ yếu hoạt động ổn định trong môi trường local hoặc LAN. Các trường hợp phức tạp hơn như NAT traversal hoặc triển khai Internet thực tế chưa được nhóm xử lý trong phiên bản này.